ChatGPT

Executive Summary

This design report details an end-to-end architecture for an **autonomous options trading agent** using Alpaca (market/trading APIs) and Serper (web search) APIs. The system divides into two main parts: **Part A (Deterministic/Processing/Execution)** and **Part B (Agentic/LLM Orchestration)**. Part A covers fixed data pipelines, risk checks, order execution, and monitoring. Part B covers AI-driven components (market, news, options agents, and decision logic) that use LLMs and tool calls to generate trade proposals. Each component is fully specified: its purpose, tasks, endpoints, data schemas, code examples, and test checklists. An implementation roadmap lists milestones and MVP scope.

Overall, the flow is: **User Configuration → Data Ingestion (Alpaca + Serper) → Universe Filter → Data Processing → (Structured State) → Agentic Layer (Market Scanner + Agents + Decision Agent) → Risk Engine → Execution → Monitoring/Journal**.

Mermaid diagrams illustrate the high-level flow and relationships. Tables enumerate required APIs, Python modules, LLM prompts and tool usage. All Alpaca and Serper APIs are mapped explicitly. Example JSON schemas and code snippets are provided for key integrations (e.g. fetching market data, Serper search, submitting an order). Security (API keys, rate limits) and error handling (retries, validation) are noted. A final roadmap prioritizes tasks.

This report is comprehensive (see “Part A: Deterministic/Processing/Execution Blocks” and “Part B: Agentic/LLM Components” below), enabling the team to implement and track each block with clear deliverables and tests.

Part A – Deterministic/Processing/Execution Blocks

1. User Configuration & Asset Pool

Purpose: Define the investment “cage” the agent can operate in. Includes user-specified boundaries (max risk, allowed strategies, etc.) and initial universe of assets to consider.

Tasks:

- **Define config schema:** e.g. YAML/JSON containing user limits (max risk per trade %, max exposure %, strategies, etc.) and asset symbols.
- **Example config JSON:**

```
{
  "assets": ["SPY","QQQ","NVDA","AAPL","MSFT","TSLA"],
```

```
  "max_risk_per_trade_pct": 1.0,
  "max_daily_loss_pct": 3.0,
  "max_open_positions": 5,
  "max_exposure_pct": 20.0,
  "min_risk_reward": 1.5,
  "max_holding_days": 10,
  "allowed_strategies":
["BullCallSpread","BearPutSpread","LongCall","LongPut"]
}
```

- **Implement loader module:** Read and validate config (e.g. `UserConfig` Python class) on startup. Ensure values are within sane ranges.
- **Deliverables/Tests:**
- ✅ Schema validation unit tests (invalid JSON raises error).
- ✅ Load config on startup with sample files.

Endpoints/Modules: No external calls here. Use Python modules: e.g. `pydantic` or custom classes for config.

Security/Notes: Store API keys separately (environment variables or vault), not in config.

---

2. Data Ingestion (Alpaca Market Data + Serper News)

Purpose: Fetch raw data required for analysis: historical and real-time market data for stocks and options; news data via web search.

Sub-tasks:
1. **Alpaca Market Data API:**

- Instantiate Alpaca **Market Data** clients: e.g.

```
from alpaca.data.historical.stock import StockHistoricalDataClient
from alpaca.data.requests import StockBarsRequest
from alpaca.data.historical.option import OptionHistoricalDataClient
from alpaca.data.requests import OptionChainRequest
stock_client = StockHistoricalDataClient(key, secret, sandbox=True)
option_client = OptionHistoricalDataClient(key, secret, sandbox=True)
```

- Fetch equity bars/quotes for symbols in universe:

```
bars = stock_client.get_stock_bars(
    StockBarsRequest(symbol_or_symbols=assets, timeframe="1Day",
                     start="2026-01-01", end="2026-08-23", limit=500)
)
```

• Fetch option chains for each underlying of interest:

```
from alpaca.data.enums import OptionsFeed
from alpaca.trading.enums import ContractType
chain = option_client.get_option_chain(
  OptionChainRequest(
    underlying_symbol="NVDA",
    feed=OptionsFeed.INDICATIVE,  # free delayed
    type=ContractType.CALL,
    expiration_date_gte="2026-09-01",
    expiration_date_lte="2026-12-31",
    strike_price_gte=100.0,
    strike_price_lte=300.0
  )
)
```

Cite: This pattern is documented in Alpaca's tutorial【18†L389-L397】.

• **Alpaca Trading API (for contracts):**

• Use TradingClient to get option contract metadata (type, strike, status):

```
from alpaca.trading.client import TradingClient
from alpaca.trading.requests import GetOptionContractsRequest
from alpaca.trading.enums import ContractType
trading_client = TradingClient(key, secret, paper=True)
resp = trading_client.get_option_contracts(
    GetOptionContractsRequest(
      underlying_symbols=["NVDA"],
      expiration_date_gte="2026-09-01",
      expiration_date_lte="2026-12-31",
      strike_price_gte="100",
      strike_price_lte="300",
      type=ContractType.CALL,
      limit=100
    )
)
contracts = resp.option_contracts
```

Cite: Alpaca docs example【18†L415-L423】.

• **Serper News API:**

• Use Serper's Google Search API to gather news/sentiment on assets. Example:

```
import requests
SERPER_KEY = "<api_key>"
headers = {"X-API-KEY": SERPER_KEY, "Content-Type": "application/json"}
params = {"q": f"{symbol} latest news 2026", "hl": "en", "gl": "us"}
news_resp = requests.post("https://google.serper.dev/search", json=params,
headers=headers)
news_data = news_resp.json()
```

This returns JSON with search results. *Cite: Serper usage from community code【28†L169-L178】【28†L186-L195】.*

- Optionally, follow top news links (if desired) via the browser tool or request page content for deeper analysis.

- **Scheduling/Frequency:**

- Ingest new data at fixed intervals (e.g. every market open, or periodically) using a scheduler (cron or async loop).

- Respect rate limits: Alpaca limits ~200 req/min【31†L383-L392】; implement caching and backoff. Serper free tier limits (e.g. ~2500 req/day) must be monitored.

Deliverables/Checklist:

- [ ] Alpaca API clients correctly fetch data for sample tickers.
- [ ] Serper API returns search JSON for test queries.
- [ ] Mock and real data integration test: verify data shape.
- [ ] Rate-limit handling: simulate 429 and retry logic test.

Endpoints/Modules Table:

Data/ Action	API Endpoint / Call	Python Module/Class
Equity bars (historical)	`GET /v2/stocks/bars` (via Alpaca SDK StockHistoricalDataClient.get_stock_bars) 【17†L274-L283】	`alpaca.data.historical.stock.StockHistoricalDataCl`
Equity quotes/ trades	`GET /v2/stocks/quotes` / `.../ trades`	`get_stock_quotes()`, `get_stock_trades()`
Option chain (snapshot)	`GET /v1beta1/options/chain` (OptionChainRequest)【18†L389-L397】	`alpaca.data.historical.option.OptionHistoricalData`

Data/ Action	API Endpoint / Call	Python Module/Class
Option bars (historical)	`GET /v1beta1/options/bars`	`OptionHistoricalDataClient.get_option_bars()`
Option trades	`GET /v1beta1/options/trades`	`OptionHistoricalDataClient.get_option_trades()`
Option contracts (metadata)	`GET /v2/options/contracts` (GetOptionContractsRequest)【18†L415-L423】	`alpaca.trading.client.TradingClient.get_option_con`
Assets list	`GET /v2/assets` (GetAssetsRequest)	`TradingClient.get_all_assets`【45†L12-L16】
Serper Web Search	POST `https://google.serper.dev/search` (body: q, hl, gl)【28†L169-L178】【28†L186-L195】	`requests.post` or LLM tool `serper.search(query)`
(Optional) News scraping	Web pages via `browser` tool (if needed)	(use agent `browser.get(page_url)`)

Data Schemas:

- *Raw Market Bars:*

```
{ "AAPL": [ {"t":"2026-08-22T13:30:00Z","o":175.2,"h":176.0,"l":174.8,"c":
175.6,"v":30.5e6}, ... ] }
```

- *Option Chain Entry:* (from Alpaca OptionChainResponse)

```
{
  "symbol": "NVDA230915C00150000",
  "underlying_symbol": "NVDA",
  "strike_price": 150.0,
  "contract_type": "CALL",
  "expiration_date": "2026-09-15",
  "bid": 3.5,
  "ask": 3.6,
  "last_price": 3.55,
  "implied_volatility": 0.24,
  "delta": 0.55,
  "gamma": 0.05,
  ...
}
```

- *Serper Search Response:*

```
{
  "organic": [
    {"position":1, "title":"NVDA Q2 Earnings Results",
"link":"https://...", "snippet":"..."},
    {"position":2, "title":"Nvidia raises guidance...", ...}
  ],
  "knowledgeGraph": { ... }
}
```

Error handling: check `response.status_code` (e.g. 200 OK, or 429 retry). Use try/except around HTTP calls.

---

3. Deterministic Universe Filter

Purpose: Quickly eliminate assets not meeting hard criteria, so AI agents only consider promising candidates. This speeds up analysis and reduces API use.

Sub-tasks:

- Loop over assets in pool and apply filters:
- **Liquidity:** e.g. average daily volume > threshold (e.g. 100k shares) or bid-ask spread < threshold.
- **Volume:** recent volume surge or minimum volume baseline. Use data from `get_stock_bars` or `get_stock_quotes`.
- **Price:** exclude very low-priced (<$10) or very high (>$1000) if desired.
- **Options availability:** call option_chain for asset; ensure at least one liquid contract exists. For example, ensure Open Interest > 1000 or bid>0 for some nearby strikes.
- **Spread:** optionally check stock bid-ask spread via latest quote (ensuring spread ratio small).
- **User constraints:** e.g. if user only allowed ETFs (SPY, QQQ) or certain sectors.

Example (pseudo-code):

```
eligible = []
for sym in asset_pool:
    bars = stock_client.get_stock_bars(StockBarsRequest([sym],
timeframe="1Day", limit=5))
    avg_vol = np.mean([bar.v for bar in bars[sym]])
    if avg_vol < MIN_VOL: continue
    latest_quote = stock_client.get_stock_quotes(...)
    spread = (latest_quote.ask - latest_quote.bid) / latest_quote.ask
    if spread > MAX_SPREAD: continue
    chain =
option_client.get_option_chain(OptionChainRequest(underlying_symbol=sym,
```

```
feed=OptionsFeed.INDICATIVE))
    if not any(c["open_interest"] > 1000 for c in chain[sym]): continue
    eligible.append(sym)
```

- Implementation: Put this logic in a **UniverseFilter** class/module. The filter returns a subset (e.g. 30 of 100 assets).

Deliverables/Checklist:

- [ ] Unit tests covering each filter criterion (e.g. synthetic low-volume asset is rejected).
- [ ] Filtered asset list matches expectations for test data.

Completion Criteria: A deterministic filter produces the expected candidate list (human-verified on sample data).

Integration: This filter runs after data ingestion and user config, before heavy processing.

---

4. Data Processing Layer

Purpose: Transform raw data into structured features needed by agents. Compute technical indicators, option metrics, normalize news, etc.

Sub-tasks (Data Processing Module):

- **Technical Indicators:** Compute e.g. moving averages, RSI, MACD, ATR using recent bars. (Use libraries like `ta` or custom).
- Input: raw OHLCV for each symbol.
- Output fields: e.g. `{"sma20": 150.2, "sma50": 145.8, "rsi14": 62.3, "atr14": 2.5}`.
- **Volatility:** Calculate price volatility (e.g. standard deviation of returns) over past N days.
- **Returns:** Compute daily return percentages, momentum (e.g. 5-day change).
- **Liquidity Metrics:** Normalize volume relative to average.
- **Option Greeks/IV:** From option_chain, extract **IV** (implied volatility) and **Greeks** (Delta/Gamma/Theta) for contracts. Already returned by OptionChain API. Compute any custom measure (e.g. wide vs. narrow spread).
- **News Normalization:** From Serper results:
- Extract headlines/snippets about the symbol.
- Apply simple sentiment analysis or mapping (positive/negative score) – could use an LLM or keyword heuristics.
- Identify catalysts or event keywords (e.g. “earnings”, “FDA approval”, “merger”).
- Format as structured info:

```
{
  "news": [
    {"source":"CNBC","title":"Nvidia Q2 beats
estimates","sentiment":"positive","impact":"high"},
```

```
    {"source":"WSJ","title":"Semiconductor industry growth
slows","sentiment":"neutral","impact":"medium"}
  ]
}
```

- **Structured State Construction:** Combine all the above into a unified JSON per candidate symbol. For example:

```
{
  "symbol": "NVDA",
  "price": 180.45,
  "trend": {"sma20": 178.2, "sma50": 173.5, "rsi14": 68},
  "volatility": {"daily_std": 0.018, "atr": 4.2},
  "volume": {"today": 34e6, "avg20": 25e6},
  "options": [
    {"strike": 180, "exp": "2026-09-15", "bid":3.5, "ask":3.6, "iv":0.24,
"delta":0.55},
    ...
  ],
  "news": [
    {"headline": "Nvidia raises guidance on AI demand",
"sentiment":"positive", "confidence":0.92}
  ]
}
```

- **Implementation:** Python modules/classes like `IndicatorCalculator`, `NewsNormalizer`, `OptionsMetricsExtractor`. Each should be unit-tested (e.g. known input yields correct RSI).

Deliverables/Checklist:

- [ ] Functions computing each indicator (unit tests against known values).
- [ ] Integration test: raw data ➔ structured JSON (manual inspection of fields).
- [ ] Example schemas (as above) documented.

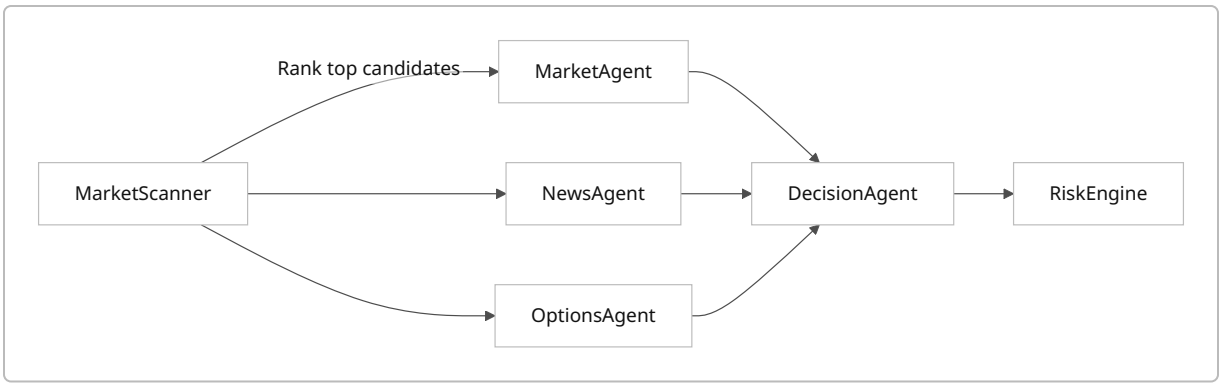
Completion Criteria: The structured state JSON contains all required fields (trends, vol, option metrics, news sentiment) and matches sample expected schema.

---

5. Agentic Layer – Market Scanner + Research Agents

This block is where LLM agents and tool calls operate. It is split into **Market Scanner**, **Market Agent**, **News Agent**, **Options Agent**, and **Decision Agent**. These components use the structured state to decide on trades.

Diagram (Agentic Flow):



5a. Market Scanner (Pre-Ranking)

Purpose: Narrow the list of candidates (from Universe Filter) based on current opportunity scores. For example, apply quick quant filters to rank e.g. top 5 assets.

Tasks:

- Define simple signals (momentum, volatility, divergence) to score each structured state. For example:
- **Momentum Score:** `recent_return / vol`
- **Trend Score:** number of bullish indicators (price above MA50, RSI>60, etc).
- Compute a score and pick top N (e.g. 5) symbols.

Checklist:

- [ ] Score calculation implemented.
- [ ] Ranking logic tested on synthetic data (verify correct ordering).

Completion: Identify a small set (3–5) of candidate symbols for detailed analysis.

5b. Market Agent

Purpose: Use LLM reasoning on market data (price patterns, indicators) to assess direction, trend, and potential strategy hints.

Tools/Calls:

- Can use the Alpaca Data tools (via MCP or python): e.g., call `alpaca.get_stock_bars(symbol, ...)` for latest data, or use the already-fetched structured data.
- LLM prompt: Provide relevant price summary (e.g. “NVDA is in an uptrend; SMA20 > SMA50; ATR rising…”). Ask the agent to confirm trend or highlight anomalies.
- Example prompt snippet:

```
Market data for NVDA: price 180.4, 5-day up 5%, 20-day SMA=178.2, 50-day SMA=174.1, RSI=68. Based on this, is NVDA trending up, down, or neutral? Explain.
```

Checklist:

- [ ] Market agent prompt design (test with few symbols).
- [ ] Ensure agent calls tools only within allowed domain.

Completion: Market agent outputs (in JSON or structured text) confidence in bullish/bearish trend for each candidate.

5c. News Agent

Purpose: Analyze relevant news content and sentiment to identify catalysts or risks.

Tools/Calls:

- **Serper Search:** Already fetched news results. Could refine search (e.g. `symbol + "stock news"`) for context.
- **Sentiment Analysis:** Use an LLM prompt or simple lexicon to classify sentiment of each headline. Optionally call a `browser` tool to open specific news URLs for more context.
- **LLM prompt:**

```
News headlines for NVDA:
1) "Nvidia raises AI forecast after strong Q2" (CNBC)
2) "AI chip competition heats up" (TechCrunch)
For each, determine positive/negative sentiment and potential impact on stock price.
```

- **Output:** e.g. `[{"headline":"...","sentiment":"positive","impact":"high"}]`.

Checklist:

- [ ] Validate that news agent identifies sentiments correctly on a few samples.
- [ ] Ensure no disallowed content extraction (just headlines and summary).

Completion: A structured list of news items with sentiment tags for each candidate.

5d. Options Agent

Purpose: Analyze options market data (IV, spreads, greeks) and propose suitable multi-leg strategies (e.g. bull/bear spreads) based on market and news inputs.

Tools/Calls:

- **Alpaca OptionChain:** Already fetched chains for candidates. The agent can filter and select strikes.
- **LLM prompt:** Provide option data summary:

```
NVDA call options chain: ATM strike 180, IV=24%, 30-day. Bull spread at
strikes 180/190 has net debit $X, max profit $Y.
Given NVDA is bullish, recommend a strategy.
```

- Possibly call a computational tool: we might use an internal function to calculate risk/reward of strategies. (E.g. Python code given by agent via function call.)
- Tools could include a custom calculator function for spreads (outside LLM).

Checklist:

- [ ] Ensure Options Agent picks realistic strikes (in- or near-the-money) and calculates greeks.
- [ ] Test suggested strategy makes sense (e.g. calls for bullish, puts for bearish).

Completion: For each candidate, Options Agent produces a structured strategy proposal (e.g. `{"type":"BullCallSpread","legs":[...],"details":{...}}`).

5e. Decision Agent

Purpose: Aggregate Market/News/Options agents' outputs and decide **(trade or no trade)**, plus strategy details. This uses LLM reasoning, weighted by confidence and risk profile.

Inputs:

- Structured state for each top candidate, including market trend, news sentiment, option strategy suggestion.
- Portfolio status (open positions, cash).

LLM Prompt Example:

```
You are the Decision Agent. For each candidate asset (with market and news
analysis, and proposed option strategy), determine whether to TRADE or PASS. If
trading, output: asset, direction (bull/bear), strategy, strike, expiration,
position size. Must consider:
- Capital allocation (≤5% portfolio per trade, ≤20% total exposure).
- Risk constraints (max 1% account risk).
Portfolio details: cash $100k, 1 open position NVDA bull spread risk $200.
Candidates: (structured data follows).
Make a structured JSON output.
```

Tools:

- Could call a portfolio lookup tool: e.g. `alpaca.get_account()` to see equity and buying power.
- Could call risk engine (see next section) to simulate position size.

Checklist:

- [ ] Validate LLM outputs JSON against schema (e.g. must include `trade:` or `no_trade`).
- [ ] Check with a few dummy states (should PASS if R/R too low, etc.).

Completion: Decision Agent outputs either a trade proposal or no-trade for each candidate, with confidence. The output should be machine-readable JSON.

---

6. Risk Engine (Deterministic Rules)

Purpose: Apply strict quantitative risk checks on any proposed trade. This ensures compliance with user config before execution.

Sub-tasks:

- **Position Sizing:** Given `max_risk_per_trade_pct` (e.g. 1%) and account balance, calculate maximum loss allowed (e.g. $1000).
- **Compute Proposal Risk:** For the proposed option trade, simulate max loss. For example, for a bull call spread: max loss = debit paid. For a long call: max loss = premium.
- **Check Limits:**
- Proposed trade risk $\leq$ max allowed.
- Position size (notional or contracts) adjusted so risk $\approx$ max allowed.
- **Risk/Reward Ratio:** If user requires min R/R (e.g. 1.5:1), verify.
- **Exposure/Exposure:** Projected portfolio exposure (if new trade opens) $\leq$ max allowed percentage.
- **Buying Power:** Compare required margin/premium vs account buying_power.
- **Open Positions Count:** If exceeding max open positions, reject additional trade.
- **Daily Loss Limit:** Calculate running daily P/L; reject if adding more trades would exceed.
- **Output:** `PASS` or `REJECT`. If `PASS`, output finalized order parameters (size, price). If `REJECT`, record reason.

Endpoints/Modules:

- **Trading API:** `trading_client.get_account()` to read cash, `get_all_positions()` to include existing exposure.
- **Tool calls:** Might call to Alpaca to check buying_power.
- Use Python for calculation.

Checklist:

- [ ] Unit tests for each rule (simulate account values).
- [ ] Test a sample trade flow through risk engine and order is approved/blocked as expected.

Completion: Only if all checks pass do we proceed. Otherwise, the trade is logged as "rejected – reason".

---

7. Order Construction & Validation

Purpose: Build the final order request and validate before sending.

Sub-tasks:

- **Order Builder:** Using the Decision Agent's proposal and adjusted sizes, construct Alpaca `OrderRequest` objects. For example:

```
from alpaca.trading.requests import MarketOrderRequest
order_data = MarketOrderRequest(
   symbol="AAPL",
   qty=10,                    # or use `notional=1000.0` for fractional
   side=OrderSide.BUY,
   time_in_force=TimeInForce.GTC
)
order = trading_client.submit_order(order_data=order_data)
```

Or for spreads, use multi-leg orders via `AlgoOrderRequest` (if supported) or multiple leg orders.

- **Order Validator:** Double-check parameters: symbol, qty, price (if limit), expiration, contract status (ensure not expired). Confirm account ID (for Broker API if needed).
- **Checklist:**
- [ ] Order JSON matches strategy semantics.
- [ ] Example: For a bull call spread, ensure SELL leg has strike above BUY leg, same exp.
- **MCP/CLI Integration:** Optionally, use the Alpaca MCP tool if integrating with an LLM workflow. For example, one could call `alpaca.create_order(...)` via MCP interface; however for code implementation, using the Python SDK as above is acceptable.

Completion: Constructed order(s) ready for execution.

---

8. Execution (Alpaca Trading API)

Purpose: Submit the approved order(s) to Alpaca (paper trading) and handle response.

Steps:
1. **API Call:** Using Alpaca `TradingClient.submit_order` as in prior example【44†L179-L187】【44†L188-L194】. If multi-leg is needed, see Alpaca bracket/condor API.

2. **Response Handling:** Capture order ID and initial status. Confirm submission success (200 OK).
3. **Error Handling:**

- On failure (network or non-200), retry a few times (with backoff).
- If 429 (too many requests), back off as per docs【31†L383-L392】.
- If error indicates no buying power or invalid order, log and abort trade.

Checklist:

- [ ] Example order submission code works in paper environment (tested manually).
- [ ] Simulated order (approved by Risk) results in entry in `get_orders()`.
- [ ] Error/resend logic tested (e.g. force a 400 or 429).

Completion: Order is in Alpaca's system (paper account). Save the `order_id` for monitoring.

---

9. Position Monitor & Exit Logic

Purpose: After entry, continuously monitor open positions for exit conditions: profit target, stop loss, or time/market-based exit.

Sub-tasks:

- **Tracking:** Poll Alpaca for fills and position status. Use `trading_client.get_all_positions()` periodically (e.g. every 15 min).
- **Profit/Loss Checks:** Calculate unrealized P/L. If P/L $\geq$ target or $\leq$ -stop threshold, trigger exit.
- **Time-based Exit:** If holding period exceeds user's `max_holding_days`, exit at market.
- **Market Conditions:** If decision agent's thesis invalidates (e.g. news sentiment turns negative), close position.
- **Exit Execution:** Use `submit_order` to close (e.g. reverse the original legs). For options, use `close_all_positions()` or individual `cancel_orders()`.
- **Logging:** Every exit decision and action logged into the trade journal.

Checklist:

- [ ] Simulate a position and ensure monitor triggers exit at target.
- [ ] Test a timed exit (mock clock).

Completion: Positions are closed according to rules, with reasons logged.

---

10. Trade Journal & Performance Analytics

Purpose: Record each trade from decision to exit, enabling review and analytics.

Schema (example JSON record):

```
{
  "trade_id": "UUID-1234",
  "symbol": "NVDA",
  "strategy": "BullCallSpread",
  "entry_time": "2026-08-23T15:30:00Z",
  "entry_legs": [{"symbol":"NVDA230915C00150000","action":"BUY","qty":
10,"price":3.50},...],
  "exit_time": "2026-08-30T11:15:00Z",
  "exit_legs": [{"symbol":"NVDA230915C00150000","action":"SELL","qty":
10,"price":5.20},...],
  "profit_loss": 1700.0,
  "reason": "Profit target reached"
}
```

- **Implementation:** Write records to a database or file (e.g. JSONL), including all relevant fields. Use `logging` for step-by-step logs (prompts, agent outputs).
- **Analytics:** Aggregate P/L, win-rate, drawdowns. Possibly produce reports (tables/graphs) as part of dashboard.

Checklist:

- [ ] Journal entries correctly capture key fields for a sample trade.
- [ ] Logging around decision and risk checks present in logs.

Completion: Full trade lifecycle is logged; analytics script can summarize at least simple metrics (total trades, P/L).

---

11. Security and Rate-Limit Notes

- **API Keys:** Secure keys (never hard-code; use env vars). Limit exposure in logs.
- **Rate Limits:** Alpaca max ~200 req/min【31†L383-L392】. Batch requests (e.g. bulk bars requests) when possible. Handle 429 by pause/retry. Serper also has query rate; monitor usage (fail fast if over limit).
- **Error Handling:** For each external call, wrap in try/except. Log exceptions. Use retry libraries or custom logic (exponential backoff).
- **Secrets:** Serper key usage should be limited to news searches, not passed to LLM prompts.

---

12. Testing and Integration

Unit Tests: For each module/class (indicator calc, filters, risk engine rules, order builder).
Integration Tests: End-to-end on paper account: feed mock data to agents and simulate a trade cycle.
E2E Demo Checklist (Hackathon):

- [ ] System starts, loads config, and initial filtering works.
- [ ] Agents propose a trade on a known scenario (e.g. NVDA uptrend) and execute in paper account.
- [ ] Risk engine enforces user limits (e.g. block trade that violates max risk).
- [ ] A trade journal entry is generated for each trade.
- [ ] Logging output shows each step (agent prompt, decisions, order placement).

Mermaid Flow Diagram:

```
flowchart TD
    UC[User Configuration\n(assets, risk limits)] --> AP[Alpaca+Serper Data]
    AP --> UF[Deterministic Filter]
    UF --> DP[Data Processing\n(Indicators, Volatility,\nNews Normalization)]
    DP --> MS[Market Scanner]
    MS --> MA[Market Agent]
    MS --> NA[News Agent]
    MS --> OA[Options Agent]
    MA & NA & OA --> DA[Decision Agent]
    DA --> RE[Risk Engine]
    RE -->|PASS| OB[Order Builder/Validator]
    RE -->|REJECT| JR[Trade Journal (rejected)]
    OB --> EX[Execute (Alpaca API)]
    EX --> PM[Position Monitor]
    PM --> JR[Trade Journal (closed)]
    JR --> AN[Performance Analytics]
```

Part B – Agentic/LLM Components and Orchestration

A. LLM & Tool Integration Design

- **Language Models:** Use a capable LLM (e.g. GPT-4) for Market/News/Options/Decision agents.
- **Tool Interfaces:**
- **Alpaca Tools:** Define tools (via MCP or direct function calls) for market data retrieval, order submission, portfolio info.
- **Serper Tool:** A search tool that calls Serper API.
- **Prompting:** Each agent gets a carefully crafted prompt and JSON-like memory state to process.
- **Orchestration:**

- A controller (Python script) calls each agent sequentially or in parallel, passing inputs and capturing outputs.
- Agents use a structured prompt that constrains their output format (JSON).

LLM Prompts & Tool Calls

- **Market Agent Prompt:**

```
You analyze stock price behavior. Given recent price and indicators:
{structured_trend_data}, decide if the trend is BULLISH, BEARISH, or
NEUTRAL. Tools available: alpaca.get_stock_bars(symbol, timeframe). Output:
{"direction": "...", "confidence": 0.85}.
```

- **News Agent Prompt:**

```
You analyze news headlines. Given headlines: {list_of_news_headlines},
classify each as positive/negative/neutral. Output: [{"headline": "...",
"sentiment": "positive"}].
```

Tool call example: The prompt might list headlines retrieved by Serper search for that symbol.

- **Options Agent Prompt:**

```
You are an options strategist. Given underlying {symbol} is
{market_direction} and implied volatility {IV}, recommend a trade strategy.
Consider available strikes and expirations. Output JSON with fields: asset,
strategy_type, legs (symbol/strike/exp/side), estimated max_loss and
max_profit.
```

Tool use: Provide example tools the LLM can call like `alpaca.get_option_chain(symbol, filters)`.

- **Decision Agent Prompt:** (as shown above). Emphasize JSON structure and risk rules.
- **Example Tool Call Interface:** (as passed to LLM if using e.g. LangChain or MCP).

```
{ "tool": "alpaca_get_bars", "args": {"symbol": "NVDA", "timeframe": "1D",
"limit": 5} }
```

LLM output schema tests: For each agent, validate JSON shape.

B. Agent Workflow Orchestration

Flow:
1. **Market Scanner selects candidates.**
2. **Parallel Agent Calls:** For each candidate:

- Call **Market Agent**, **News Agent**, **Options Agent** (can be parallel threads).
- **Collect results** from all agents.
- **Decision Agent** receives combined state for each and produces proposals.

Use a sequence tool or orchestrator code to handle this.

Checklists:

- [ ] Agent prompts include context and exact expected JSON format (validate with a few runs).
- [ ] Ensure LLM does not hallucinate (tie it tightly to input data).
- [ ] Logging: capture prompt+response for audit (for talk through demo).

C. Integration with MCP/CLI

- The Alpaca **MCP (Meta-Command Platform) server** can expose Alpaca APIs as "tools" to the LLM (e.g. in ChatGPT or Claude). For code, using the Python SDK is easier.
- For demonstration: show an example of calling the Alpaca CLI from Python:

```
# using subproc to simulate CLI usage:
alpaca order create -s SPY -q 10 -S BUY -T GTC
```

But in code we use `trading_client.submit_order`.

- **Routing orders:** If using MCP, the LLM could output a tool call `alpaca.create_order(...)` instead of code. We would capture that tool invocation and convert to `submit_order` call.

Checklist:

- [ ] Demonstrate CLI usage (if needed) with sample order via MCP.
- [ ] Integration test: issue an order via python + MCP key, verify in paper account.

D. Monitoring, Logging, Trade Journal (Implemented)

Covered in Part A: logs for each step, full audit trail, and final trade journal entry.

Testing Plan

- **Unit Tests:** For all deterministic functions (indicators, filters, risk).
- **Mock Agents:** Simulate agent outputs to test orchestration flow.
- **Integration Tests:**

- **Data Flow:** Given sample market/news data, run through entire pipeline to ensure outputs align.
- **Live Paper Test:** Deploy on an Alpaca paper account with known test assets, observe an end-to-end cycle (including trade execution).
- **E2E Demo:** Prepare a script that runs one session from start to finish, highlighting each block's output.

Implementation Roadmap

1. **Setup & Data Layer (Effort: Low)** – Configure Alpaca API keys, fetch sample bars and options (use example code above). Validate connections.
2. **Deterministic Modules (Medium)** – Implement Universe Filter, Indicators, Risk Engine. Write unit tests.
3. **Order Execution (Low)** – Use TradingClient to place and cancel dummy orders in paper. Test normal and error paths.
4. **Monitoring & Journal (Low)** – Basic loop to poll positions and record P/L; write to JSON lines file.
5. **Agentic Pipeline (High)** – Develop prompts, tool interfaces, and LLM orchestration for Market/News/Options agents and Decision agent.
6. **Integration & CLI (Medium)** – Wire everything together, perhaps integrate Alpaca MCP or CLI for tool calls.
7. **UI/Analytics (Optional for MVP)** – Basic output (print statements or simple dashboard) of active positions and performance.

Minimal Viable Product (MVP):

- Focus on one strategy (e.g. bull call spreads) and a small asset pool (e.g. 3 symbols).
- Ensure end-to-end working: scanning ➔ propose ➔ risk check ➔ execute ➔ exit ➔ log.
- Advanced features (multiple strategies, GUI) can be deprioritized.

By following this detailed architecture and checklist, the team can systematically implement each block, verify functionality, and integrate into the full autonomous options trading agent.
